

プログラミング②

(基本構文と for 文と if 文)

2026 年度高 2 情報

2026/05/09

机に貼ってある番号に従って着席・課題を出す方は前の机に
PC の準備を (PASS:Est1922) ・ paiza.io/ja にアクセス (C#)

変数の名づけのルール

変数の名前でだめなもの

- 予約語
 - int, double などのデータの型 (種類) を決める言葉
 - if や for など既に特別な制御に関係があるもの
- 演算子
 - + や -, *, / などの演算記号
- 数字から始まるもの `1st_score` などはダメ
- 全角文字や半角スペース・全角スペースを含むもの (全角文字もできるようになっているが, 避けるべきである)

変数の名づけのルール

よくある書き方 (camelCase)

- 英単語 (ローマ字) をつなげて表す
- 2 単語目以降の先頭の文字は大文字

例 kamokuld

- 使うプログラミング言語や職場 (研究所) などによってもルールが違う. 先に確認しよう

C#の主なデータ型 - 1

まず覚えるべき基本の型

型の名前	データの種類	例
int	整数 (小数点なし)	10, -5, 0
double	小数 (実数)	3.14, -0.5
string	文字列 (文章など)	"Hello", "武蔵"
bool	真偽値 (2 択)	true, false

※ 先週の授業の通り、プログラム中では文字列は " " (ダブルクォーテーション) で囲む

C#の主なデータ型 - 2

知っておくと便利な型（応用編）

型の名前	データの種類	特徴・用途
long	大きな整数	int（約21億）を超える時
float	小数 (浮動小数点を利用)	doubleより精度が低い 処理が軽い(メモリが少ない)
decimal	高精度な小数	お金の計算など誤差NGな時
char	1文字	'A' や 'あ' など1文字限定
byte	0～255の整数	画像データなどの特殊な用途

※ 余裕のある人は覚えてみよう (ただし授業では用いない)

for 文と if 文

プログラムの基本構造 (p97-p99)

すべてのプログラムは順次構造, 分岐構造 (選択構造), 反復構造の組み合わせによって構成される.

- 順次構造とは, 上から下へ記述された順に処理を実行するという構造である (先週確認済み)
- 分岐構造とは, 条件の真偽に応じて処理を実行する構造である (if 文など)
- 反復構造とは類似した処理を繰り返す構造である (for 文や while 文など)

for 文を実行してみよう

```
public class Add{  
    public static void Main(){  
        int i=0;  
        int sum=0;  
        //i=0からiが10以下の間だけ  
        //iを足すという操作を繰り返す  
        for(i=0;i<=10;i=i+1){  
            sum=sum+i;  
        }  
        System.Console.WriteLine(sum);  
    }  
}
```

for 文の構造

for 文の基本構造

```
for (初期化式; 条件式; 変化式){  
    // 繰り返したい処理  
}
```

- 1 初期化 ($i = 0$) : 最初の一回だけ実行
- 2 条件判定 ($i \leq 10$) : 中身を実行する「前」にチェック
 - 条件を満たさない (偽) 場合は、その時点で終了
- 3 ループの中身 ($sum = sum + i$) を実行
- 4 変化式 ($i = i + 1$) を中身の実行が終わった後に実行
- 5 2 の条件判定に戻る

while 文を実行してみよう

```
public class Add{  
    public static void Main(){  
        int i=0;  
        int sum=0;  
        while(i<=10){  
            sum=sum+i;  
            i=i+1;  
        }  
        System.Console.WriteLine(sum);  
    }  
}
```

while 文の構造

while 文の基本構文

```
while (条件式){  
    // 繰り返したい処理  
}
```

- 1 条件判定 ($i \leq 10$) : 中身を実行する「前」にチェック
 - 条件を満たさない（偽）場合は、その時点で終了
- 2 ループの中身を実行
- 3 1 の条件判定に戻る

for 文と while 文について

共通点と相違点

- for 文も while 文も 1 回も実行されない可能性がある
一度は実行する「do-while 文」というものもある。興味があるものは調べてみよ。
- for 文は回数の指定に長けている
- while 文は条件の指定に長けている (故に無限ループが起こりやすい)

if文を実行してみよう-1

```
public class Judge{  
    public static void Main(){  
        int num=10; //numが10だと上のみが実行され, 10以外にす  
        ると下のみが実行される  
        if(num==10){  
            System.Console.WriteLine("numは10です");  
        }  
        if(num!=10){  
            System.Console.WriteLine("numは10ではありません")  
        };  
    }  
}
```

if文を実行してみよう-2

```
public class Judge{  
    public static void Main(){  
        int num=10;  
        if(num>=10){  
            System.Console.WriteLine("numは10以上です");  
        }  
        else{  
            System.Console.WriteLine("numは10未満です");  
        }  
    }  
}
```

if文を実行してみよう-3

```
public class Judge{
    public static void Main(){
        int num=7;
        if(num>=10){
            System.Console.WriteLine("numは10以上です");
        }
        else if(num>=5){
            System.Console.WriteLine("numは5以上10未満で
す");
        }
        else{
            System.Console.WriteLine("numは5未満です");
        }
    }
}
```

if文の基本構文-1

条件に合致した時に処理をしたいとき

```
if(条件式){  
    // 処理  
}
```

条件に合致した時と合致しないときにそれぞれ処理があるとき

```
if(条件式){  
    // 処理1  
}  
else{  
    // 処理2  
}
```

if文の基本構文-2

処理が3つ以上あるとき

```
if(条件式){  
    // 処理1  
}  
else if(条件式){  
    // 処理2  
} // else ifは何回繰り返してもよい  
else{  
    // その他の場合の処理  
} // elseはなくてもよい
```


for 文などを途中で止めたいとき

例

```
public class judgePrime {  
    public static void Main() {  
        int i = 0;  
        for(i = 0; i <= 5; i = i + 1){  
            System.Console.WriteLine(i);  
            if(i==2){  
                break;  
            }  
        }  
    }  
}
```

このように break を使うことで繰り返し文はループを抜け出すことができる。

条件式の書き方（比較演算子）

値を比べるための記号

記号	意味	例
<code>==</code>	等しい	<code>num == 10</code>
<code>!=</code>	等しくない	<code>num != 10</code>
<code>></code>	より大きい	<code>num > 10</code>
<code><</code>	より小さい	<code>num < 10</code>
<code>>=</code>	以上	<code>num >= 10</code>
<code><=</code>	以下	<code>num <= 10</code>

数学の「=」はプログラミングでは「==」と表す
プログラミングの「=」は代入の意味

複雑な条件（論理演算子）

「かつ」や「または」で条件をつなぐ

記号	意味	例
&&	かつ (AND)	<code>a >= 0 && a <= 10</code>
	または (OR)	<code>a == 1 a == 9</code>
!	～ではない (NOT)	<code>!(a == 0)</code>

- **かつ**：左右の条件が両方とも正しい (true) ときに実行
- **または**：どちらか一方でも正しければ実行